Querious: A Vector-Based Information Storage and Retrieval System

Syed Ali Jaseem
Department of Computer Science
University of South Dakota, Vermillion, United States
Email: syedali.jaseem@coyotes.usd.edu

***Abstract*—This paper presents Querious, a document-centric question answering system built around core principles of information storage and retrieval. It supports natural language queries over user-uploaded PDF documents through a complete retrieval-augmented generation (RAG) pipeline that includes document ingestion, preprocessing, chunking, embedding, semantic indexing, and similarity-based retrieval. Each document is processed using a sentence-aware chunking strategy with configurable size and overlap to balance context preservation and embedding quality. Text chunks are embedded into 3072-dimensional vectors and stored in a MongoDB Atlas collection with a cosine similarity-based vector index. When a user submits a query, it is embedded into the same space and matched against stored vectors using approximate nearest neighbor search to retrieve the most relevant chunks. The system improves responsiveness by streaming results through server-sent events, reducing latency from roughly 45–60 seconds to around 7 seconds. Additional optimizations such as pre-filtering and quality presets allow users to adjust retrieval depth and speed. Querious demonstrates how classical information retrieval concepts like chunking, indexing, similarity measures, and ranking can be adapted to modern vector-based retrieval while balancing accuracy, latency, and storage efficiency.**

***Index Terms*—Information Storage and Retrieval, Vector Search, Semantic Indexing, Document Chunking, Retrieval-Augmented Generation**

I. Introduction

Information retrieval systems have traditionally relied on keyword-based search, where inverted indexes map terms to document identifiers and ranking functions determine relevance. While effective for exact matching, these methods struggle when users phrase queries differently from how information appears in documents. This vocabulary mismatch problem limits the effectiveness of purely lexical retrieval. Vector-based semantic search addresses this issue by representing both documents and queries as dense vectors within a shared embedding space. Instead of matching exact terms, similarity is measured by geometric distance between vectors, allowing the system to identify semantically related content even when the wording differs. Combined with large language models in a retrieval-augmented generation (RAG) setup [3], this approach enables natural language question answering over document collections while grounding responses in actual source material.

This paper presents Querious, an information storage and retrieval system developed as a course project to explore these concepts in practice. The system allows users to upload PDF documents, which are processed through a pipeline of parsing, chunking, embedding, and indexing. When a user submits a natural language question, the system retrieves relevant document chunks using vector similarity search and generates a grounded, source-attributed response. The paper focuses on the design and optimization of the storage and retrieval pipeline. Key areas include chunking strategies and their effect on retrieval quality, vector embedding and indexing for semantic search, similarity measures and approximate nearest neighbor search, pre-filtering and scope-based retrieval optimization, and latency reduction using streaming and quality presets. Figure 1 shows a high-level overview of the system architecture, illustrating the flow from document ingestion to storage, indexing, and query processing.

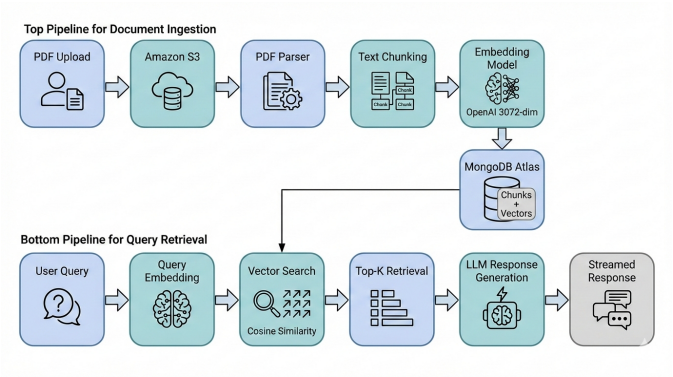

Fig. 1: High-level architecture of Querious showing the document ingestion and query retrieval pipelines.

II. Background and Related Work

A. Information Storage and Retrieval Systems

Information retrieval (IR) systems are designed to locate and return relevant information from a collection of documents in response to a user query. Classical IR systems rely on an inverted index, a data structure that maps terms to the documents containing them [1]. When a query is submitted, the system looks up each term in the index to retrieve candidate documents, which are then ranked using a scoring function.

The probabilistic retrieval model, represented by BM25, remains one of the most widely used ranking functions [5]. It considers term frequency, inverse document frequency, and

document length normalization to calculate relevance scores. The BM25 scoring function is given by:

$$\text{Score}(D,Q) = \sum_{t \in Q} IDF(t) \cdot \frac{f(t,D) \cdot (k_1 + 1)}{f(t,D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{\text{avgdl}})}$$

where $f(t, D)$ is the frequency of term t in document D, $|D|$ is the length of the document, and k_1 and b are tuning parameters. While effective for keyword matching, these methods require overlap between query and document terms, which can cause issues when users phrase queries differently from how information appears in documents.

To address this limitation, research in semantic retrieval has focused on representing meaning beyond exact term matches.

B. Vector-Based and Semantic Retrieval

Vector-based retrieval represents text as dense vectors within a continuous embedding space. Neural embedding models, often based on transformer architectures, encode semantic meaning into fixed-dimensional vectors [2]. Documents with similar meanings cluster closely together in this space, even when they use different wording.

Similarity between vectors is usually measured using cosine similarity, computed as:

$$\text{sim}(A,B) = \frac{A \cdot B}{\|A\|\|B\|}$$

Cosine similarity focuses on the angle between two vectors rather than their magnitude, making it well-suited for comparing text embeddings where document length should not affect similarity [6].

The main strength of vector-based retrieval is its ability to find semantically related content. For instance, a query about "vehicle maintenance" can return documents discussing "car repair" even without shared terms. However, this approach introduces computational overhead, since embeddings must be generated for all documents and queries, and efficient nearest neighbor search requires specialized vector index structures.

III. Methodology

A. System Architecture

1) Overall System Design: Querious follows a modular system architecture that separates document ingestion from query-time retrieval. These two pipelines operate independently while sharing core components such as the embedding model and vector index. This separation enables documents to be processed asynchronously without blocking interactive query handling.

The system consists of four primary components: a FastAPI backend that orchestrates all operations, Amazon S3 for raw document storage, MongoDB Atlas for structured metadata and vector indexing, and a React frontend for user interaction. Document ingestion is coordinated using Inngest, an event-driven workflow engine that supports multi-step execution with automatic retry handling and state tracking [9].

Figure 1 in Section I illustrates the high-level data flow through both pipelines. The ingestion pipeline processes uploaded documents through parsing, chunking, embedding, and storage. The retrieval pipeline embeds user queries, performs vector search, ranks relevant chunks, and generates responses using a large language model.

2) Document Ingestion Pipeline: When a user uploads a PDF document, the system first stores the raw file in Amazon S3 using a unique key derived from the user and access scope. A corresponding document record is created in MongoDB with its status set to *pending*, capturing metadata such as filename, file size, checksum, and scope. An ingestion event is then triggered asynchronously.

The ingestion workflow proceeds through several stages. The system retrieves the PDF from S3 and parses it using LlamaIndex's PDFReader, which extracts textual content while preserving page boundaries [10]. The extracted text is segmented into chunks using a sentence-aware splitting strategy to avoid breaking sentences mid-way.

Each chunk is embedded using OpenAI's text-embedding-3-large model, producing a 3072-dimensional dense vector [8]. The resulting chunks, along with embeddings and metadata such as page number and chunk index, are stored in the *chunks* collection in MongoDB. Upon successful completion of all steps, the document status is updated to *ready*, indicating that it is available for retrieval.

3) Document Processing and Chunking: Chunking strategy directly influences retrieval effectiveness. Chunks that are too small may lack sufficient context, while overly large chunks may combine unrelated topics and reduce relevance when only a portion matches a query.

The system uses LlamaIndex's SentenceSplitter with a chunk size of 1000 characters and an overlap of 200 characters. This configuration ensures that chunks align with natural sentence boundaries while preserving contextual continuity across adjacent segments. The overlap reduces the risk of important information being split across chunk boundaries.

Chunk size selection was guided by empirical testing. Smaller chunks of approximately 500 characters improved precision for narrow queries but often lacked sufficient context. Larger chunks of approximately 2000 characters provided richer context but resulted in embeddings that were overly general. A size of 1000 characters with 200 characters of overlap provided the best balance between retrieval accuracy and response efficiency for the evaluated document types.

4) Vectorization and Indexing: Each text chunk is converted into a dense vector using OpenAI's text-embedding-3-large model. The resulting 3072-dimensional embeddings encode semantic meaning, enabling retrieval based on vector similarity rather than exact keyword overlap.

To reduce ingestion latency, embeddings are generated in batches at the document level rather than per chunk. The resulting vectors are stored alongside chunk text and metadata in the *chunks* collection.

MongoDB Atlas Vector Search provides the indexing infrastructure. Cosine similarity is used as the distance metric,

as it compares vector orientation rather than magnitude and is well suited for text embeddings of varying lengths. A filter on *document_id* is applied before vector comparison to restrict candidates to relevant documents.

Internally, MongoDB Atlas Vector Search employs the Hierarchical Navigable Small World (HNSW) algorithm to enable efficient approximate nearest-neighbor search [7]. This allows scalable retrieval performance for collections containing thousands of stored chunks.

5) Data Model and Relationships: The system uses three primary collections in MongoDB: *documents*, *chunks*, and *document_scopes*. Together, these collections support document management, semantic retrieval, and scope-based access control.

The *documents* collection stores metadata for each uploaded file, including filename, S3 key, checksum for deduplication, and processing status. The *chunks* collection contains processed text segments with embeddings and links each chunk to its parent document through *document_id*. The *document_scopes* collection implements a many-to-many relationship between documents and access scopes such as chat sessions or projects.

Chunk identifiers are generated deterministically using UUID5 based on the document identifier and chunk index. This design supports idempotent ingestion, ensuring that repeated ingestion updates existing records rather than creating duplicates. The resulting data model enables consistent storage, efficient retrieval, and strict scope-based access control.

B. Information Storage Design

1) Raw Document Storage: The system uses Amazon S3 for storing original PDF documents [11]. This design separates raw file storage from structured metadata and indexed content, offering several advantages for an information retrieval system.

Object storage such as S3 is well suited for large binary files that are written once and read infrequently. Raw PDFs are accessed only during initial ingestion or re-processing when the chunking or embedding strategy changes. Storing them outside the database avoids bloating MongoDB with binary data and keeps the vector index focused on searchable content.

Each document is stored with a unique S3 key derived from the user ID and a generated document identifier. This key structure enables straightforward access control at the storage layer and simplifies cleanup during deletion. The system also computes a SHA-256 checksum of each uploaded file, which serves two purposes: detecting duplicate uploads before storage and verifying file integrity during retrieval.

2) Metadata and Structured Storage: MongoDB Atlas serves as the primary database for structured data, including document metadata, text chunks with embeddings, and scope relationships [4]. The document-oriented model aligns well with variable metadata structures and supports flexible schema evolution.

Three collections form the core data model:

The *documents* collection stores metadata for each uploaded file. Key fields include the filename, S3 key for retrieval, checksum for deduplication, processing status, file size, and timestamps. A unique index on the checksum field prevents duplicate documents from being ingested, while an index on status supports efficient filtering of documents by processing state.

The *chunks* collection stores processed text segments along with their vector embeddings. Each chunk record includes the parent document ID, chunk index for ordering, page number for citation, text content, and a 3072-dimensional embedding array. The vector search index on the embedding field enables semantic retrieval, while a compound index on document ID and chunk index supports ordered retrieval of all chunks for a given document.

The *document_scopes* collection implements access control by linking documents to scopes. A scope represents a context in which documents are accessible, such as a chat session or a project. This many-to-many relationship allows a single document to be shared across multiple contexts without duplication, while queries can be restricted to documents within a user's current scope.

TABLE I: MongoDB Index Configuration

Collection	Index	Purpose
documents	checksum (unique)	Prevent duplicate uploads
documents	status	Filter by processing state
chunks	embedding (vector)	Semantic similarity search
chunks	document_id	Filter chunks by document
chunks	(document_id, chunk_index)	Ordered chunk retrieval
document_scopes	(scope_type, scope_id)	Scope-based document lookup
document_scopes	document_id	Cascade delete support

3) Storage Design Decisions: The hybrid storage architecture, with S3 for raw files and MongoDB for structured data and vectors, represents a deliberate tradeoff between storage efficiency, retrieval performance, and operational simplicity.

Storing raw PDFs in S3 rather than MongoDB avoids the 16 MB document size limit and lowers database storage costs for large files. S3's tiered storage also allows infrequently accessed documents to be moved to cheaper storage classes over time. The tradeoff is added complexity in coordinating between two storage systems during upload and deletion.

Storing embeddings directly in MongoDB alongside chunk text, instead of using a separate vector database, simplifies the architecture by avoiding synchronization between multiple systems. MongoDB Atlas Vector Search provides sufficient performance for the expected collection sizes (thousands to tens of thousands of chunks), and its integrated filtering, which pre-filters by document ID before vector comparison, is essential for scope-based retrieval.

The scope-based access model adds a layer of indirection but enables critical functionality. Without it, each chat or project would require its own copy of uploaded documents, leading to storage duplication and inconsistent updates. The *document_scopes* collection allows efficient resolution of which documents are searchable in a given context, reducing the vector search space and improving both performance and relevance.

C. Search and Retrieval Pipeline

1) Query Processing: When a user submits a natural language question, the system processes it through a pipeline that transforms the text into a vector representation and retrieves semantically relevant document chunks.

The query text is embedded using the same model applied during document ingestion, OpenAI's text-embedding-3-large [8]. Using identical embedding models for both documents and queries ensures that both reside in the same vector space, enabling meaningful similarity comparisons. The resulting 3072-dimensional query vector serves as the search key for retrieval.

Before executing the vector search, the system resolves the document scope. Based on the current context, such as a chat session or a project, the system determines which documents are accessible. This resolution queries the *document_scopes* collection to obtain a list of document identifiers to include in the search. If no documents are associated with the current scope, the system returns early with an appropriate response rather than executing an empty search.

2) Vector Search Execution: The core retrieval operation uses MongoDB Atlas Vector Search with the `$vectorSearch` aggregation stage [4]. The search is configured with the query embedding, a limit on the number of returned results (top-k), and a pre-filter that restricts candidates to chunks belonging to accessible documents.

The aggregation pipeline structure is shown conceptually below:

```
{
  "$vectorSearch": {
    "index": "vector_index",
    "path": "embedding",
    "queryVector": <query_embedding>,
    "numCandidates": 100,
    "limit": 10,
    "filter": {
      "document_id": {"$in": <accessible_doc_ids>}
    }
  }
}
```

The `numCandidates` parameter controls how many candidates the HNSW index examines before returning the top-k results. A higher candidate-to-result ratio improves recall at the cost of increased search time. The system uses 100 candidates for a limit of 10, providing a 10:1 ratio that balances accuracy and performance for typical workloads.

The pre-filter on `document_id` is critical for both performance and correctness. By restricting the search space to accessible documents before vector comparison, the system avoids returning chunks from unauthorized sources. This filter leverages the indexed filter field configured in the vector search index, ensuring efficient execution even with large candidate sets.

3) Ranking and Relevance Scoring: MongoDB Atlas Vector Search returns results ranked by cosine similarity score, with higher scores indicating greater semantic similarity to the query. For normalized vectors, similarity scores range from 0 to 1, where 1 indicates identical direction in the embedding space.

The system retrieves the top-k chunks based on this ranking. The value of k is determined by the selected quality preset, as summarized in Table II.

TABLE II: Quality Presets and Retrieval Parameters

Preset	Top-K	Estimated Tokens	Use Case
Quick	5	∼1,500	Fast answers with minimal context
Standard	10	∼3,000	Balanced speed and response quality
Thorough	20	∼6,000	Deeper retrieval for higher accuracy
Deep	35	∼10,500	Comprehensive context coverage
Max	50	∼15,000	Maximum context retrieval

Higher top-k values retrieve more context and improve recall at the cost of increased token consumption and response latency. The preset system exposes this trade-off to users, allowing them to select an appropriate balance based on their needs.

4) Context Assembly: After retrieval, the system assembles the context provided to the language model for response generation. Each retrieved chunk includes the text content along with metadata such as the source document filename and page number.

Chunks are concatenated in descending order of similarity score, with delimiters indicating source boundaries. This assembled context, combined with the user's original question and recent conversation history, forms the final prompt.

Conversation history is incorporated using a sliding window approach. The system retains the most recent messages up to a configurable limit of 10 messages or 4000 tokens, providing conversational continuity without exceeding context limits. Older messages are excluded from the prompt but remain stored for reference.

The final prompt structure follows a consistent pattern:

1) A system instruction defining the assistant's role and citation requirements
2) Retrieved context with source attribution
3) Recent conversation history, if applicable
4) The current user question

This structure encourages responses grounded in retrieved context and discourages unsupported generation.

5) Response Generation and Streaming: The assembled prompt is sent to the language model API, which generates a response conditioned on the retrieved context. The system uses DeepSeek's API for response generation due to its balance of response quality and cost.

To improve perceived responsiveness, responses are streamed using Server-Sent Events rather than waiting for complete generation. The streaming pipeline emits events at multiple stages:

- Status events indicating progress through search and analysis stages

- Source events delivering the list of referenced documents
- Chunk events streaming partial response text as it is generated
- A final event signaling completion with token usage information

This staged streaming approach provides immediate feedback to users, demonstrating ongoing processing rather than presenting an idle interface during the 10 to 25 second generation period.

D. Optimization Techniques

1) Latency Reduction Strategies: Response latency in a RAG system accumulates across multiple stages, including query embedding, vector search, context assembly, and language model generation. The system implements several optimizations to reduce both actual and perceived latency.

Rather than searching the entire chunk collection and filtering results afterward, the system pre-filters candidates using the indexed `document_id` field. This reduces the effective search space from potentially thousands of chunks across all users to only those belonging to accessible documents. For a typical user with 10 to 20 documents, this optimization can reduce the candidate set by orders of magnitude compared to a global search.

The most significant improvement in perceived latency comes from streaming response delivery. Without streaming, users would wait 45 to 60 seconds with no feedback while the system searched, retrieved, and generated a complete response. With Server-Sent Events, the system provides stage indicators within 2 to 3 seconds, source information within 5 to 7 seconds, and begins streaming response text within 7 to 10 seconds. The complete response still requires 12 to 25 seconds depending on complexity, but continuous feedback substantially improves the user experience.

During document ingestion, all chunks are embedded in a single API call rather than embedding each chunk individually. This batching reduces API overhead and latency during ingestion, although it does not affect query-time performance.

Allowing users to select retrieval depth provides a direct latency and accuracy tradeoff. The Quick preset retrieves fewer chunks and produces faster responses suitable for simple factual queries. The Standard preset balances performance and completeness, while higher presets retrieve more context at the cost of increased token usage and longer generation times.

2) Accuracy and Relevance Improvements: Retrieval accuracy depends on embedding quality, chunk granularity, and the effectiveness of ranking mechanisms.

Splitting text at sentence boundaries rather than fixed character lengths preserves semantic coherence within chunks. Chunks that end mid-sentence tend to embed poorly because incomplete thoughts weaken semantic signal. The 200-character overlap further ensures that context near chunk boundaries is preserved.

The use of 3072-dimensional embeddings enables finer semantic discrimination than lower-dimensional alternatives. While this increases storage requirements to approximately 12 KB per chunk, it improves the system's ability to distinguish between closely related concepts, which is particularly valuable for technical or specialized document collections.

Including page numbers and source filenames with each retrieved chunk enables verifiable citations. This allows users to trace responses back to original documents, enhancing transparency and trust in the system.

Incorporating recent conversation history in the prompt supports contextual follow-up queries. Questions such as "What about the second option?" rely on prior discussion to be interpretable. A sliding window approach balances contextual continuity with token constraints.

3) Trade-Off Analysis: Every optimization involves tradeoffs. Table III summarizes the primary tradeoffs in the system design.

TABLE III: Design Tradeoffs in Querious

Decision	Benefit	Cost
Larger chunk size	More context per chunk	Lower precision for narrow queries
Chunk overlap	Context preservation	Increased storage (20% overhead)
3072-dimensional embeddings	Better semantic precision	Higher storage cost per chunk
Pre-filtering	Faster search and access control	Scope resolution overhead
Higher top-k	Improved recall	Increased token usage and latency
Streaming	Improved perceived latency	Added implementation complexity

The combination of high-dimensional embeddings and chunk overlap increases storage requirements but directly improves retrieval quality by preserving context and enabling finer semantic discrimination.

The quality preset system exposes the latency and accuracy tradeoff to users. Retrieving more chunks improves recall but increases response time. Users can prioritize speed for time-sensitive queries or depth for comprehensive analysis.

Storing embeddings directly in MongoDB alongside chunk text simplifies the system architecture compared to maintaining a separate vector database. This approach works well at the current scale but may require migration to a dedicated vector store if the dataset grows to millions of chunks.

IV. Evaluation

A. Performance Metrics

The system was evaluated across several performance dimensions relevant to information retrieval, including response latency, retrieval relevance, and user-perceived responsiveness.

End-to-end response latency was measured from query submission to complete response delivery. Table IV summarizes the observed latency across different quality presets.

TABLE IV: Response Latency by Quality Preset

Preset	Chunks	Time to First Token	Total Time
Quick	5	∼5s	10–15s
Standard	10	∼7s	12–20s
Thorough	20	∼8s	18–25s
Deep	35	∼10s	22–30s
Max	50	∼12s	28–40s

Time to first token represents when the user first sees partial response content and is the primary indicator of perceived responsiveness. Total time reflects complete response generation and varies with response length and retrieval depth.

Prior to implementing Server-Sent Events streaming, users experienced 45 to 60 seconds of wait time with no feedback. After streaming was introduced, the time to first visible response dropped to approximately 7 seconds for the Standard preset, reducing perceived latency by roughly 85 percent. Complete response delivery also improved by 50 to 70 percent, now ranging from 12 to 25 seconds depending on retrieval depth.

The vector search stage itself completes in under 500 milliseconds for typical collections of up to 10,000 chunks. Most query latency is attributable to language model generation rather than retrieval. Pre-filtering by document scope reduces the effective search space to approximately 100 to 1,000 chunks per query, keeping vector search times consistently low.

B. Qualitative Evaluation

Retrieval relevance was assessed through manual inspection of query results across a range of document types and question formulations.

The system demonstrated strong semantic understanding. Queries phrased differently from the source text still retrieved relevant chunks. For example, a query about authentication methods successfully retrieved content discussing login procedures and access control mechanisms without requiring keyword overlap.

All responses included accurate source attribution with document names and page numbers. This traceability allowed users to verify claims directly in the original documents and proved essential for building trust in system outputs.

When relevant information spanned multiple documents, the system retrieved chunks from each source and synthesized a coherent response. The language model effectively integrated context across documents while maintaining clear attribution.

For queries about topics not covered in the uploaded documents, the system appropriately indicated that the information was unavailable in the current context. This behavior aligned with system instructions and reduced the risk of hallucinated responses.

Table V summarizes the qualitative assessment across key retrieval capabilities.

TABLE V: Qualitative Capability Assessment

Capability	Observation
Semantic matching	Strong; handles synonyms and paraphrasing
Multi-document synthesis	Effective; combines sources with attribution
Source citation	Accurate; page-level references provided
Unanswerable queries	Appropriate; acknowledges missing information
Follow-up questions	Supported; maintains conversational context

V. Discussion

The design and implementation of Querious provided practical insight into how classical information retrieval concepts can be applied within a modern vector-based architecture.

1) Design Strengths: The combination of high-dimensional embeddings and sentence-aware chunking consistently produced relevant retrieval results for targeted queries. Pre-filtering by document scope reduced the search space and improved both performance and relevance by excluding inaccessible documents before vector comparison. Server-Sent Events streaming transformed the user experience from a 45–60 second wait to progressive feedback beginning within approximately 7 seconds, significantly improving perceived responsiveness without altering overall generation time.

The quality preset system proved effective by exposing the retrieval depth tradeoff directly to users. For quick factual lookups, lower presets provided faster responses with sufficient context. For research-oriented queries requiring comprehensive coverage, higher presets retrieved broader context at the cost of increased latency.

2) Limitations: Chunk-based retrieval inherently loses global document structure. Very broad queries, such as requests for the main conclusions of a multi-chapter document, sometimes retrieved individually relevant snippets without capturing the overarching summary. This behavior reflects the limitations of flat chunk-based retrieval, where higher-level document organization is not explicitly modeled.

The fixed 1000-character chunk size worked well for narrative and explanatory content but occasionally split structured elements such as tables or lists. This reduced the usefulness of some retrieved segments. Adaptive chunking strategies that respect document structure and layout could mitigate this issue.

3) Implications: This project demonstrates that classical information retrieval concepts such as indexing, ranking, and similarity measurement remain highly relevant in modern vector-based systems. The core tradeoffs between precision and recall, speed and accuracy, and storage and retrieval quality persist even when representations shift from sparse term vectors to dense semantic embeddings.

Rather than replacing traditional IR principles, vector-based retrieval extends them, offering improved semantic matching while preserving the fundamental challenges and design considerations that define information retrieval systems.

VI. Supporting System Components

A. Backend Design

The backend is implemented using FastAPI, a Python web framework that supports asynchronous request handling and automatic API documentation [12]. All request and response payloads are validated using Pydantic models, which enforce type constraints and prevent malformed data from propagating into the ingestion and retrieval pipelines [13]. This validation layer improves system robustness and simplifies error handling.

Long-running operations, including document ingestion and embedding generation, are executed asynchronously using Inngest. This design decouples user-facing API requests from

background processing, preventing request timeouts and ensuring consistent system responsiveness during intensive workloads.

B. Frontend Overview

A React-based frontend was developed to support document upload, chat-based querying, and real-time response streaming for testing and demonstration purposes [14]. The frontend serves as a thin interaction layer and is not a focus of this work.

VII. Future Work

Several directions could further extend the retrieval capabilities demonstrated in this project. While the current architecture supports document collections with thousands of chunks, scaling to substantially larger corpora would require additional investigation. Potential approaches include sharding the vector index or migrating to a dedicated vector database, as well as exploring hybrid retrieval methods that combine sparse keyword-based techniques with dense vector search to improve performance for queries that benefit from exact term matching.

Retrieval accuracy could also be improved through second-stage reranking, where a cross-encoder model re-scores the top retrieved candidates to refine relevance ordering. Additionally, adaptive chunking strategies that respect document structure, such as section boundaries or tables, could improve retrieval quality for complex documents. At a systems level, caching frequently used query embeddings and distributing embedding generation across multiple workers could reduce latency and ingestion time for larger workloads.

VIII. Conclusion

This paper presented Querious, an information storage and retrieval system that applies classical information retrieval principles within a modern vector-based architecture. The system implements a complete retrieval pipeline, including document ingestion, chunking, embedding, indexing, and similarity-based search, producing grounded and source-attributed responses to natural language queries.

The project demonstrated that core information retrieval concepts such as indexing, similarity measures, and relevance ranking remain central even when representations shift from sparse term vectors to dense semantic embeddings. Trade-offs between precision and recall, chunk granularity and context preservation, and retrieval depth and response latency continue to shape system design. Overall, Querious illustrates how established IR principles can be effectively applied to contemporary vector-based retrieval systems while preserving transparency, efficiency, and relevance.

Acknowledgment

The author would like to thank Dr. Rodrigue Rizk for his guidance and feedback throughout the development of this project. His insights into information retrieval concepts and system design were instrumental in shaping both the system implementation and this paper.

References

[1] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[4] MongoDB, Inc., "Atlas Vector Search," MongoDB Documentation, 2024. [Online]. Available: https://www.mongodb.com/docs/atlas/atlas-vector-search/

[5] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[6] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed Representations of Words and Phrases and their Compositionality," in *Advances in Neural Information Processing Systems*, 2013, pp. 3111–3119.

[7] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[8] OpenAI, "Embeddings," OpenAI Platform Documentation, 2024. [Online]. Available: https://platform.openai.com/docs/guides/embeddings

[9] Inngest, Inc., "Inngest Documentation," 2024. [Online]. Available: https://www.inngest.com/docs

[10] LlamaIndex, "LlamaIndex Documentation," 2024. [Online]. Available: https://docs.llamaindex.ai/

[11] Amazon Web Services, "Amazon S3 Documentation," 2024. [Online]. Available: https://docs.aws.amazon.com/s3/

[12] S. Ramírez, "FastAPI Documentation," 2024. [Online]. Available: https://fastapi.tiangolo.com/

[13] Pydantic, "Pydantic Documentation," 2024. [Online]. Available: https://docs.pydantic.dev/

[14] Meta Platforms, Inc., "React Documentation," 2024. [Online]. Available: https://react.dev/